

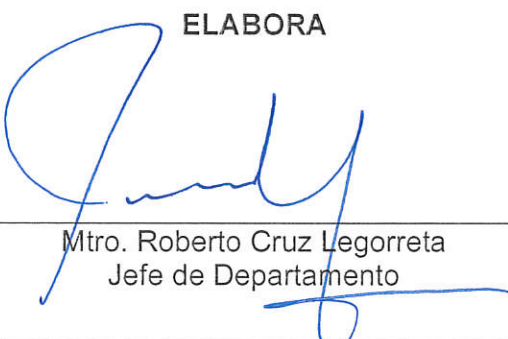


Comisión Federal de Electricidad®

**Estándares en Materia de Soluciones Tecnológicas,
Sistemas Informáticos y Bases de Datos**

**Coordinación de Servicios Tecnológicos
Gerencia de Tecnologías de Información
Subgerencia de Informática**

Versión 1.0

HOJA DE FORMALIZACIÓN			
CLAVE DEL ÁREA: K4000			COORDINACIÓN DE SERVICIOS TECNOLÓGICOS
DÍA	MES	AÑO	GERENCIA DE TECNOLOGÍAS DE INFORMACIÓN
18	06	2024	
AUTORIZA		PROPONE	
 _____ Ing. Marco Antonio López Meléndez Coordinador de Servicios Tecnológicos		 _____ Ing. Manuel Alejandro Soriano Ferreyra Gerente de Tecnologías de Información	
REVISAS		ELABORA	
 _____ Mtro. Luis Felipe Pérez Navarro Subgerente de Informática		 _____ Mtro. Roberto Cruz Legorreta Jefe de Departamento	
VIGENCIA: A partir de la fecha que señala esta Hoja de Formalización, sin exceptuar los cambios y/o modificaciones sustantivas que se presenten durante el periodo.			
OBSERVACIONES:			
CRÉDITOS:			
Ing. Erick García de la Rosa Lic. Gonzalo Toledo García Lic. Tania Maribel Zendejas García		Auxiliar Técnico Auxiliar Técnico Jefa de Oficina II	
CONTROL DE ACTUALIZACIONES			
Revisión No.	Fecha	Motivo o Causa:	Hoja (s) No.
01	18-06-2024	Creación	Todas

Índice

1.	Introducción.....	4
2.	Marco Normativo.	4
3.	Criterios Técnicos Normativos	5
4.	Arquitectura de Desarrollo	6
4.1	Flujo de Comunicación entre Capas.....	7
4.2	Arquitectura de Servidores Intranet	8
4.3	Arquitectura de Servidores Internet	9
4.4	Arquitectura de Servidores para Aplicaciones Móviles	10
5.	Estándares Tecnológicos.	11
6.	Siglas y Acrónimos	12
7.	Nomenclaturas	13
8.	Lineamientos de Programación	13
8.1.	Nomenclatura de Objetos	13
8.2.	Nomenclatura de Controles	13
8.3.	Codificación.....	13
8.4.	Manejo de Errores	15
8.5.	Consideraciones para Proyecto Web.....	16
8.6.	Consideraciones para Proyecto Windows	16
8.7.	Consideraciones para Tareas Automatizadas	16
8.8.	Consideraciones para Controladores en API	17
8.9.	Conexión a Directorio Activo	17
8.10.	Protocolo de Tiempo de Red (NTP)	19
9.	Lineamientos para Base de Datos.....	19
9.1	Nomenclatura de los objetos de las bases de datos.....	19
9.2	Tablas	21
9.3	Procedimientos Almacenados	22
9.4	Funciones.....	23
9.5	Reportes.....	24
10	Análisis de vulnerabilidades y revisión del código.....	25
11	Documentación del sistema.....	25

1. Introducción.

El presente documento tiene como objetivo especificar los estándares tecnológicos, a efecto de gobernar los desarrollos, liberaciones y mantenimientos de las soluciones tecnológicas, sistemas informáticos y bases de datos de la Comisión Federal de Electricidad (CFE) y sus Empresas Productivas Subsidiarias (EPS), y en su caso, sus Filiales (EF).

2. Marco Normativo.

LEYES

Ley de la Comisión Federal de Electricidad, vigente.

REGLAMENTOS

Reglamento de la Ley de la Comisión Federal de Electricidad, vigente.

NORMATIVIDAD INTERNA

Código de Conducta, vigente.

Código de Ética de la Comisión Federal de Electricidad, sus Unidades de Negocio, Empresas Productivas Subsidiarias y Empresas Filiales, vigente.

Estatuto Orgánico de la Comisión Federal de Electricidad, vigente.

Lineamientos en Materia de Seguridad de la Información de la CFE y sus Empresas Productivas Subsidiarias, Filiales y terceros, vigente.

Lineamientos en Materia de Servicio de Internet en la CFE, sus EPS y Filiales, vigente.

Lineamientos Internos para Identificar, Resguardar y Proteger la Información Susceptible de Considerarse como Reservada, Confidencial o Secreto Comercial de la CFE, sus EPS y Filiales, vigente.

Lineamientos para el Diseño y Administración de Sitios Web en la CFE, sus EPS, UN y en su caso EF, vigente.

Lineamientos para la Administración y el uso de las Plataformas Colaborativas de la CFE, sus EPS y, en su caso, Empresas Filiales, vigente.

Lineamientos para la Consolidación y Administración del Directorio Activo de la CFE y sus EPS y Filiales, vigente.

Lineamientos para los Sistemas Corporativos, Multiempresa y de uso específico de la CFE y sus Empresas Productivas Subsidiarias y Filiales, vigente.

Manual de Organización de la Gerencia de Tecnologías de Información, vigente.

Manual de Organización General de la Comisión Federal de Electricidad, vigente.

Políticas de Tecnologías de la Información, Comunicaciones y Seguridad de la Información de la Comisión Federal de Electricidad, Empresas Productivas Subsidiarias y, en su caso, Empresas Filiales, vigente.

Políticas y Normas para Garantizar a los Trabajadores de CFE la Protección de sus Datos Personales, vigente.

3. Criterios Técnicos Normativos

Los sistemas informáticos, aplicaciones móviles, portales y servicios tecnológicos del Corporativo, Unidades de Negocio, EPS y, en su caso, EF de la CFE deberán:

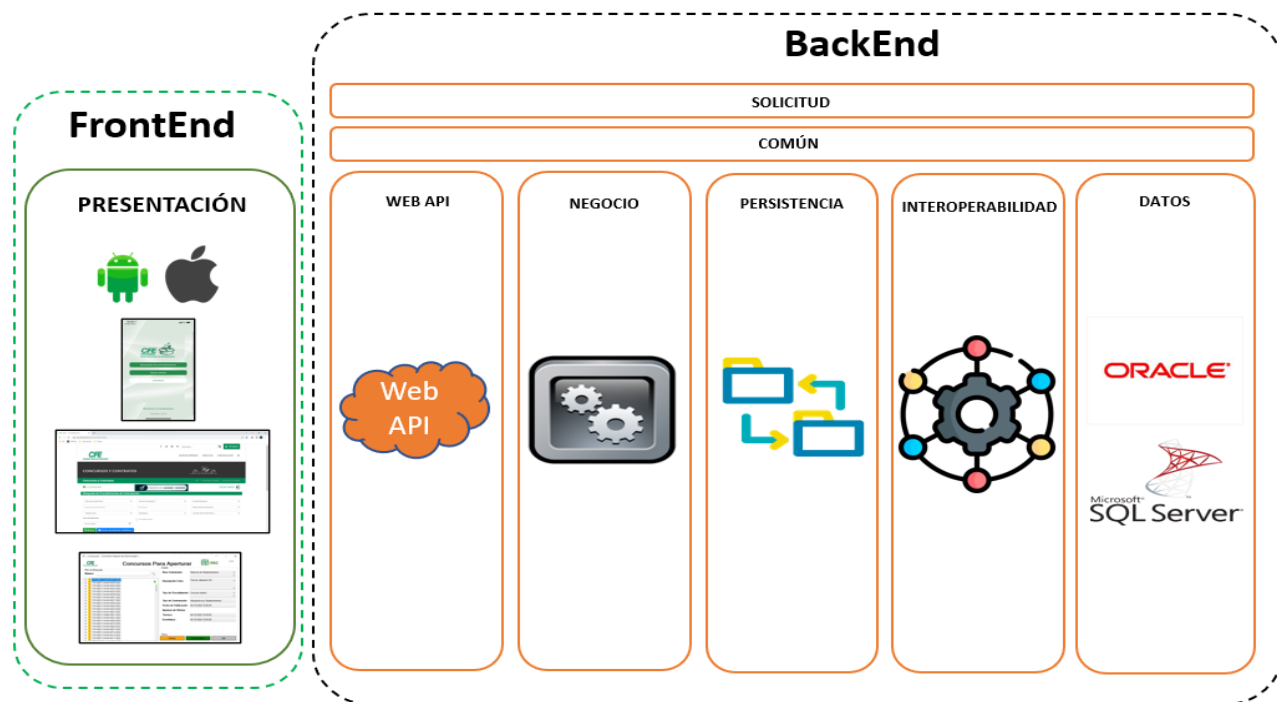
- 3.1 Hacer uso de los servicios Web de la imagen corporativa.
- 3.2 Hacer uso de los catálogos y servicios institucionales provistos por la GTI, mismos que se especifican en la sección de Servicios Tecnológicos de la Intranet Corporativa:

Normatividad TIC / Estándares Tecnológicos / Servicios y Catálogos Institucionales.pdf

- 3.3 Hacer uso de los certificados SSL emitidos por la entidad certificadora definida por la GTI. Por lo que el formato de nombres de los sitios Web será:
 - Internet: [https://\[nombre_sitio\].cfe.mx](https://[nombre_sitio].cfe.mx)
 - Intranet: [https://\[nombre_sitio\].cfemex.com](https://[nombre_sitio].cfemex.com)
- 3.4 En caso de requerir enviar notificaciones por medio del correo electrónico, estas deberán realizarse a través de los servicios provistos por la GTI; con la finalidad de optimizar recursos únicamente fungirán como alias de envío.
- 3.5 La validación de la identidad de los usuarios de CFE, sus EPS y, en su caso, EF deberá realizarse a través del Directorio Activo definido por la GTI, el cual se basa en un único dominio denominado CFE.MX.

4. Arquitectura de Desarrollo

Los sistemas informáticos que se desarrollen en y para la CFE, deberán basarse en una arquitectura conformada al menos por las siguientes capas:



- **Arquitectura general de desarrollo:** los patrones de desarrollo de código se basarán en el Modelo Vista Controlador (MVC).
- **Capa de Presentación (Web):** es la interfaz con la cual el usuario interactúa.
- **Capa Web API (API):** es la interfaz que expone los servicios que comunican a la capa de presentación y la capa de negocio.
- **Capa de Negocio (Business):** contiene las reglas de negocio y las funcionalidades definidas para el aplicativo.
- **Capa de Persistencia (Factory Data):** es la capa en la que se realizan las conexiones y se ejecutan las sentencias e instrucciones hacia la base de datos.
- **Capa de Interoperabilidad (Interop):** es la capa en la que se realizan las conexiones o comunicaciones hacia componentes y/o servicios internos o externos.
- **Capa de Datos (Entities):** es una capa transversal en la que reside el modelo y la base de datos de un aplicativo, así como los distintos contenedores en los que se almacena la información.
- **Capa de Solicitud (Request):** es una capa transversal que define el objeto de la petición en el Front End hacia el Back End.
- **Capa Común (Common):** es una capa transversal en la que se resguardan variables, constantes y funciones comunes que pueden reutilizarse en todas las capas.

Las diferentes capas se representan y relacionan de la siguiente manera:

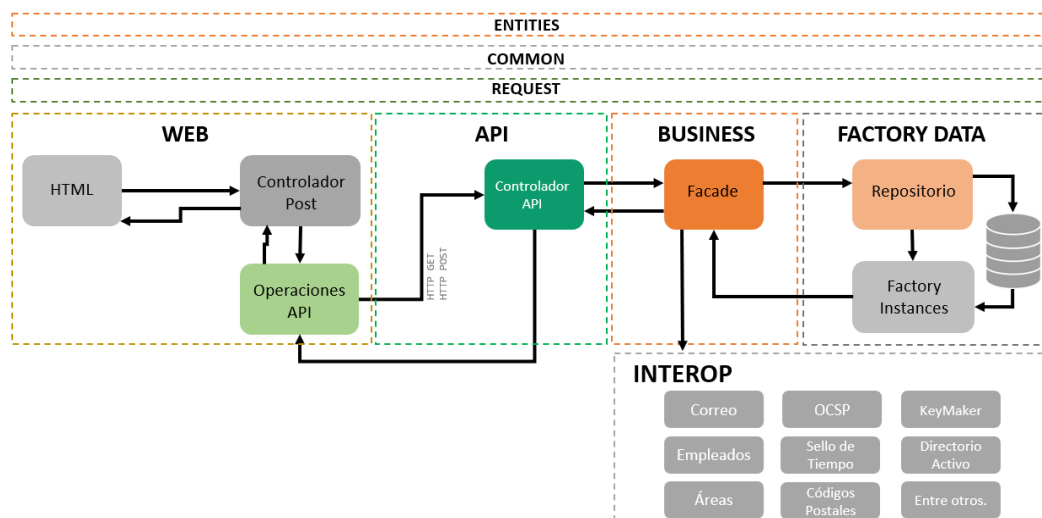
Interfaz	Capa	Nombre del proyecto
Front End	Presentación	Web
	Web API	API
Back End	Negocio	Business
	Persistencia	Factory Data
	Interoperabilidad	Interop
	Datos	Entities
	Solicitud	Request
	Común	Common

La arquitectura y distribución en capas permite separar responsabilidades, funciones y acciones proporcionando una serie de ventajas, tales como:

- Manejo de modularidad.
- Facilita la escalabilidad y el mantenimiento.
- Mayor seguridad y flexibilidad.
- Ejecución de pruebas por capa.

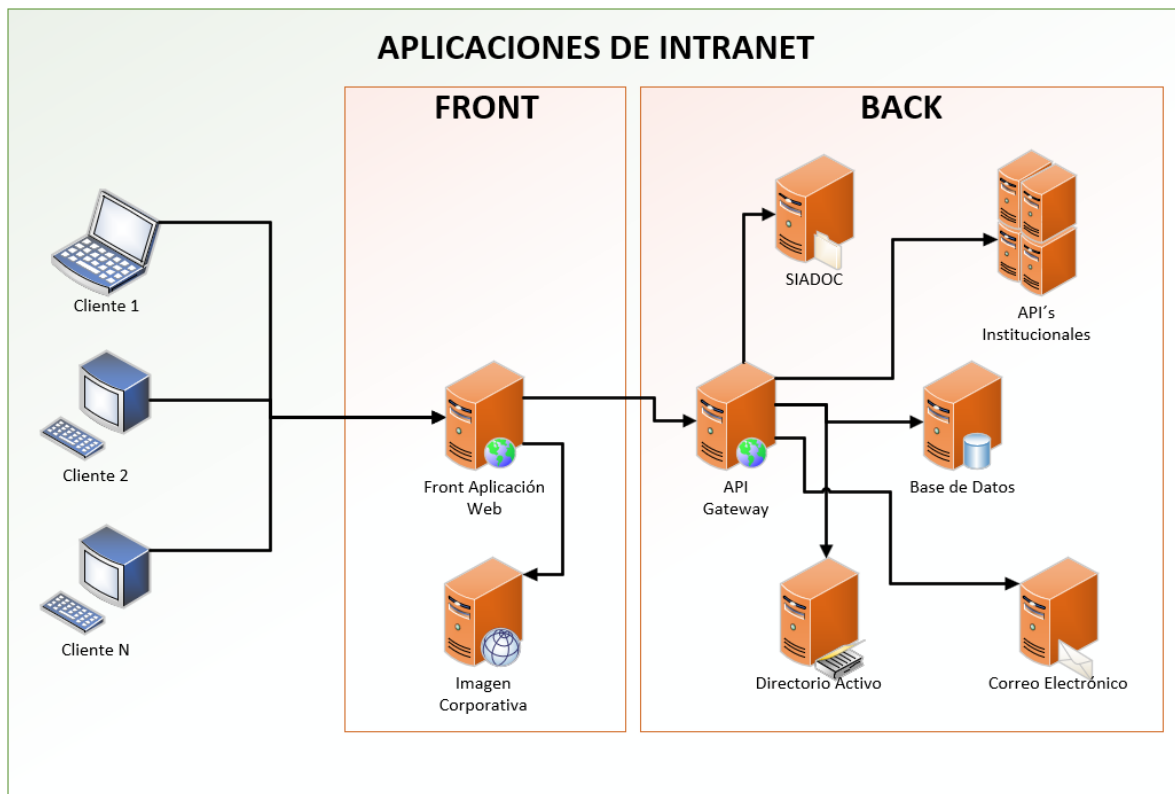
4.1 Flujo de Comunicación entre Capas

Con base en el estándar definido y siguiendo las mejores prácticas, a continuación, se muestra el flujo de comunicación entre las capas de desarrollo.



4.2 Arquitectura de Servidores Intranet

Los desarrollos que se publiquen únicamente en la Intranet deberán considerar la arquitectura distribuida que a continuación se presenta, con al menos los siguientes servidores:

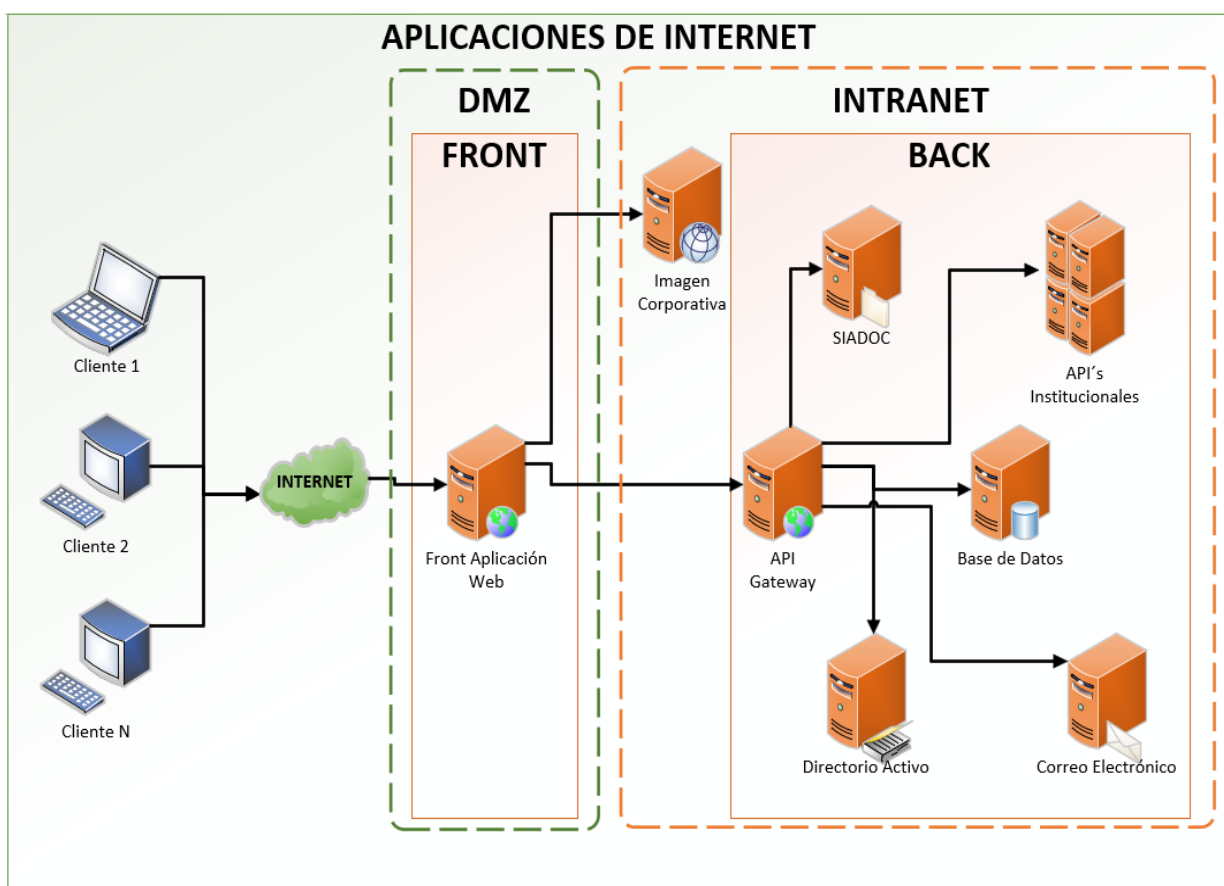


La arquitectura propuesta permite distribuir y controlar la comunicación entre los distintos componentes del servicio de Back End manteniendo homologada la comunicación y facilitando la conexión con los componentes del Front End.

4.3 Arquitectura de Servidores Internet

Todos los desarrollos que se publiquen en Internet deberán alojarse en los Centros de Datos de la CFE autorizados por la GTI, y apegarse a las POLITICAS DE TECNOLOGIAS DE LA INFORMACION COMUNICACIONES Y SEGURIDAD DE LA INFORMACION DE LA COMISION FEDERAL DE ELECTRICIDAD Y SUS EMPRESAS PRODUCTIVAS SUBSIDIARIAS Y, EN SU CASO, EMPRESAS FILIALES, así como en los LINEAMIENTOS EN MATERIA DE SEGURIDAD DE LA INFORMACIÓN DE LA CFE Y SUS EMPRESAS PRODUCTIVAS SUBSIDIARIAS, FILIALES Y TERCEROS.

Además, deberán considerar la arquitectura distribuida que a continuación se presenta, con al menos los siguientes servidores:



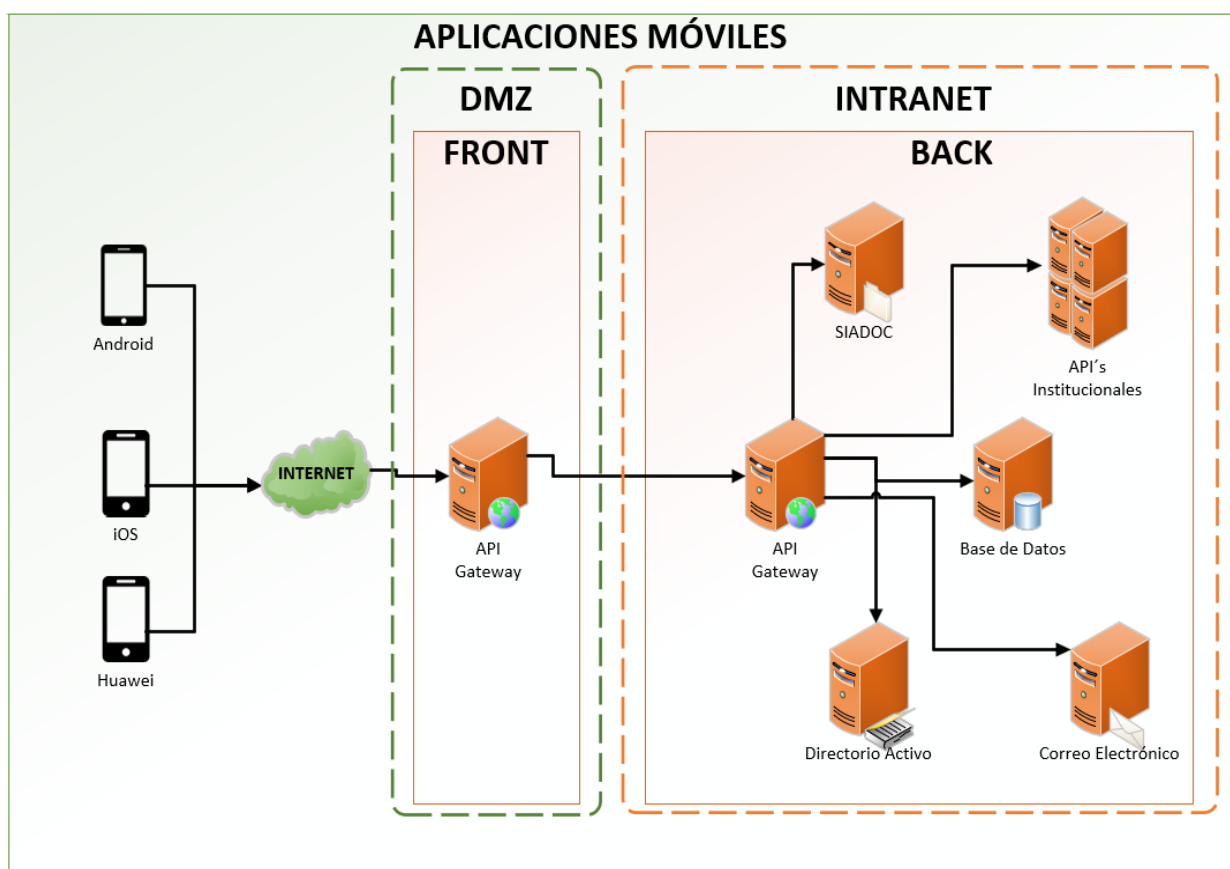
La arquitectura propuesta permite contar con mayor seguridad al no exponer en la DMZ componentes internos, tales como, bases de datos, consultas de información reservada de empleados, áreas, correos electrónicos, directorio activo, entre otros. En caso de que algún servicio requiera hospedar en la DMZ otros componentes, deberá solicitar su autorización a la GTI, con la finalidad de determinar su factibilidad y, en su caso, su autorización.

Con este tipo de arquitectura, sólo se expone el componente WEB que publica en Internet las páginas que el usuario visualiza y estas a su vez se comunican al servicio API GATEWAY, el cual se encarga de realizar la comunicación con los componentes de Intranet o bien con los componentes que se encuentran fuera de la red de CFE.

4.4 Arquitectura de Servidores para Aplicaciones Móviles

Todos los desarrollos de aplicaciones móviles que se publiquen en Internet deberán alojarse en los Centros de Datos de la CFE, y apegarse a las POLITICAS DE TECNOLOGIAS DE LA INFORMACION COMUNICACIONES Y SEGURIDAD DE LA INFORMACION DE LA COMISION FEDERAL DE ELECTRICIDAD Y SUS EMPRESAS PRODUCTIVAS SUBSIDIARIAS Y, EN SU CASO, EMPRESAS FILIALES, así como a los LINEAMIENTOS EN MATERIA DE SEGURIDAD DE LA INFORMACIÓN DE LA CFE Y SUS EMPRESAS PRODUCTIVAS SUBSIDIARIAS, FILIALES Y TERCEROS.

Los desarrollos de aplicaciones móviles que se publiquen en Internet deberán considerar la arquitectura distribuida que a continuación se presenta, con al menos los siguientes servidores:



La arquitectura anterior, permite establecer el servicio API Gateway, el cual recibe las peticiones que vienen desde una aplicación móvil a través de Internet; este API Gateway utiliza el mecanismo de autenticación por Token, el cual asegura el consumo seguro del servicio. En este sentido, una vez recibida la petición se valida el Token y desde la DMZ se realiza la consulta o petición a través del API de Intranet, la cual se encarga de ejecutar las reglas de negocio y de establecer comunicación con componentes internos o externos a CFE.

5. Estándares Tecnológicos.

5.1 Servidores:

- Utilizar preferentemente el sistema operativo Windows Server 2022 en español.
 - Previa justificación se evaluará el uso del sistema operativo Windows Server 2019 y 2016 ambas en español.
- En caso de requerir el sistema operativo Linux, utilizar preferentemente Red Hat Enterprise Linux 9.3.
 - Previa justificación se evaluará como mínimo el uso de Red Hat Enterprise Linux 9.2, 9.1, 9.0, y 8.9.
- El administrador Web de aplicaciones será el Internet Information Services (IIS) versión 10 o en su caso, Apache 2.4.
 - Previa justificación se evaluará como mínimo el uso de IIS 8 y 9.
 - Previa justificación se evaluará como mínimo el uso de Apache 2.2 y 2.0.

5.2 Base de Datos:

- En los sistemas cuya criticidad esté establecida entre baja y media, se deberá utilizar el Manejador de Base de Datos SQL Server 2022 en español.
 - Previa justificación se evaluará como mínimo el uso del Manejador de Base de Datos SQL Server 2016.
- En los sistemas cuya criticidad esté establecida como alta, se podrá utilizar el Manejador de Base de Datos Oracle 19 C.
 - Previa justificación se evaluará como mínimo el uso del Manejador de Base de Datos Oracle 18C.

5.3 Entorno de Desarrollo Integrado (IDE):

- Utilizar Microsoft Visual Studio 2022 en español.
 - Previa justificación se evaluará como mínimo el uso de Microsoft Visual Studio 2017 en español.
- Utilizar el Framework de desarrollo .Net de Microsoft en versión 8.0.
 - Previa justificación se evaluará como mínimo el uso del Framework de desarrollo .Net de Microsoft en versión 4.8.
- Utilizar para aplicaciones móviles el Framework .Net MAUI versión 8.0.

5.4 Control de Versiones de Código Fuente:

- Utilizar Azure DEVOPS, para almacenar el código y controlar sus versiones, este software lo proporcionará la GTI.

5.5 Repositorio Documental:

- Utilizar el Sistema de Administración Documental (SIADOC) como plataforma para vincular los sistemas informáticos que requieran almacenar documentos.

5.6 Tecnologías de desarrollo:

- Utilizar la arquitectura WEB API REST para el desarrollo de servicios Web.
- Utilizar los estándares definidos por la GTI para la generación de reportes y exportaciones a Excel, PDF, etc.; tales como:
 - LOCAL REPORT
 - REPORTING SERVICES
 - CLOSED XML

5.7 Herramientas de minería y limpieza de datos:

- Utilizar la herramienta Integration Services para la limpieza de información, perfilado y homologación de datos.

5.8 Herramientas de tableros de control:

- Utilizar las herramientas Web Focus y Power BI para la generación de la capa de inteligencia de negocio, tales como: tableros de control, reportes ejecutivos e informes.

- 5.9 En caso de requerir algún software adicional no contemplado en los presentes estándares, deberá solicitarse previa justificación a la Subgerencia de Informática adscrita a la GTI, para su análisis, dictaminación y en su caso, aprobación.

6. Siglas y Acrónimos

En esta sección se definen las siglas y acrónimos utilizados en el documento.

No	Definición	Descripción
1	API Gateway	A través del API Gateway se controla el acceso a los sistemas y servicios de Back End y contribuye en la optimización de la comunicación entre los clientes externos y los servicios de Back End.
2	Back End	Servidor de una aplicación de software o sitio Web incluye la base de datos y la lógica de la aplicación.
3	CFE	Comisión Federal de Electricidad.
4	Closed XML	Librería que permite la generación de documentos Excel.
5	CSS	Permite definir cómo se verán los elementos HTML en una página Web, controlando aspectos, tales como: el color, la fuente o el tipo de fuente (Cascading Style Sheets).
6	Data Warehouse	Permite almacenar grandes volúmenes de datos, procesarlos y transformarlos.
7	EF	Empresas Filiales.
8	EPS	Empresa Productivas Subsidiarias.
9	Front End	Sitio Web en el que los usuarios interactúan directamente, incluye elementos visuales, tales como: la interfaz gráfica, botones y formularios.
10	GTI	Gerencia de Tecnologías de la Información.
11	GUID	Es un identificador único global que se utiliza en programación y en sistemas informáticos para asignar identificaciones únicas a recursos (Globally Unique Identifier).
12	Integration Services	Utilería de SQL Server que permite realizar procesos de extracción, transformación y carga de datos.
13	Javascript	Es un lenguaje de secuencias de comandos que te permite crear contenido de actualización dinámica.
14	JS	Lenguaje de programación que se ejecuta en el navegador del usuario y permite manipular el contenido de la página (Java Script).
15	Local Report	Utilería que permite generar un reporte sin necesidad de que esté publicado en un servidor de Reporting Services.
16	LOG	Son archivos que registran eventos, mensajes o actividades relevantes que ocurren en un sistema.
17	Reporting Services	Utilería de SQL Server que permite la construcción de reportes.
18	SIADOC	Sistema de Administración Documental.
19	SSL	Es un protocolo de seguridad estándar utilizado para establecer una conexión cifrada entre un navegador Web y un servidor (Secure Sockets Layer).
20	Web API Rest	Es una interfaz de programación de aplicaciones que sigue los principios de la arquitectura REST (Transferencia de Estado Representacional) y se utiliza para permitir la comunicación entre sistemas, a través de la Web.

7. Nomenclaturas

En esta sección se especifican las nomenclaturas utilizadas en cada uno de los lineamientos definidos para el desarrollo de código fuente y objetos en base de datos.

No	Nomenclatura	Descripción
1	caMel	Determina que la primera letra de cada palabra debe ser mayúscula exceptuando la primera palabra. Ejemplo: nombreProveedor .
2	Mayúsculas	Determina que toda la palabra va en mayúsculas.
3	PasCal	Determina que la primera letra de cada palabra debe ser mayúscula. Ejemplo: OfertasConcurso .
4	Snake_Case	Determina que cada una de las palabras se separa por guion bajo. Ejemplo: etapa_procedimiento_ofertas .

8. Lineamientos de Programación

En esta sección se definen los lineamientos de programación para homologar el código fuente de los sistemas desarrollados por y para la CFE.

8.1. Nomenclatura de Objetos

No	Tipo	Nomenclatura	Ejemplo
1	Clase	PasCal	OfertasConcurso
2	Interface	Húngara	OfertasConcurso
3	Enumeración	PasCal	TipoProcedimiento
4	Propiedad	PasCal	ApellidoPaterno
5	Variable	caMel	nombreProveedor
6	Parámetro	caMel	numeroConcurso
7	Métodos	PasCal	BuscarConcursos
8	Variable Global	caMel	tipoOferta
9	Lista	caMel	listaUsuarios

8.2. Nomenclatura de Controles

No	Control	Prefijo	Ejemplo
1	Acordeón	ac	acOpciones
2	Botón	btn	btnGuardar
3	Campo de Fecha	dt	dtApertura
4	Campo de Lista	ddl	ddlTipoProcedimiento
5	Campo de Texto	txt	txtNombre
6	Checkbox	chk	chkSi
7	Etiqueta	lbl	lblNumero
8	Grid	grd	grdOfertas
9	Image	img	imgImagen
10	Radio Button	rdb	rdbOpciones
11	Radio List	rdl	rdlRespuestas
12	Tabla	tbl	tblConcursos
13	TextArea	txtA	txtAObservaciones

8.3. Codificación

8.3.1 Utilizar siempre los prefijos definidos para los distintos objetos y controles.

- 8.3.2 Usar nombres descriptivos por cada objeto siguiendo la nomenclatura definida.
- 8.3.3 Para usuarios de CFE, utilizar el mecanismo de autenticación por Directorio Activo.
- 8.3.4 Para usuario externos a CFE, encriptar la contraseña a través de un algoritmo no reversible.
- 8.3.5 La información sensible, se deberá encriptar a través de un algoritmo reversible.
- 8.3.6 Mantener las clases y JAVASCRIPT con un máximo de 2,000 caracteres.
- 8.3.7 Agregar el atributo especial ASP-APPEND-VERSION= "TRUE", al llamar desde el HTML un archivo JAVASCRIPT, imagen o CSS, esto nos ayuda a solventar el tema de tener que descargar la nueva versión de los archivos CSS y JAVASCRIPT, para lo cual ocupamos borrar caché o presionar el botón de actualización.
- 8.3.8 Crear una clase por cada objeto.
- 8.3.9 Utilizar puntos para separar los nombres de los proyectos siguiendo la nomenclatura siguiente CFE.SiglasProyecto.Capa, por ejemplo:
 - 8.3.9.1 **CFE.SEC.Web**
 - 8.3.9.2 **CFE.SEC.Business**
- 8.3.10 Justificar el código para mantener una estructura organizada y legible, nunca usar espacios en blanco para justificar, en su caso, usar TAB.
- 8.3.11 Usar saltos de línea entre métodos y declaraciones de variables, por ejemplo:
 - 8.3.11.1 Usar una línea en blanco entre:
 - 8.3.11.1.1 Métodos.
 - 8.3.11.1.2 Propiedades.
 - 8.3.11.1.3 Sección de variables locales y la primera sentencia de un método.
 - 8.3.11.1.4 Secciones lógicas dentro de un método.
- 8.3.12 Cuando una expresión no quepa en una sola línea de código, dividirla de acuerdo con los siguientes principios:
 - 8.3.12.1 Nueva línea después de una coma:


```
MetodoNombreLargo(parametro1, parametro2,
                      parametro3, parametro4, parametro5)
```
 - 8.3.12.2 Nueva línea después de un operador aritmético:


```
var resultado = a * b / (c - g + f) +
                1. + z;
```
- 8.3.13 Agregar un comentario en cada línea del código fuente explicando el motivo de la línea.
- 8.3.14 En cada clase se deberá agregar la definición completa de la funcionalidad, agregar un encabezado por método:


```
/// <summary>
/// Descripción de la funcionalidad del método, ¿para qué es?
/// </summary>
/// <param name="idTipoContrato">Descripción del parámetro</param>
/// <returns>Describir lo que regresa el método</returns>
```
- 8.3.15 Usar sentencias de manejo de excepción para controlar los errores y registrarlos en el Log. Por ejemplo:


```
try
{
    // Código a ejecutar
```

- ```
}
catch(Exception ex)
{
 // Manejo del error
}
finally
{
 // Funcionalidad adicional que debe ser ejecutada al final de cada método
 en cualquier eventualidad
}
}
```
- 8.3.16 Utilizar regiones para agrupar los distintos métodos de un controlador, tomando como base la funcionalidad o proceso.
- 8.3.17 Para los valores constantes, definir una clase en la capa COMMON y declararlos en mayúsculas en regiones por grupos, por ejemplo:
- ```
#region Estatus  
    public const int ESTATUS_ACTIVO = 1;  
    public const int ESTATUS_INACTIVO = 2;  
    public const int ESTATUS_BLOQUEADO = 3;  
#endregion
```
- 8.3.18 Emplear mascarillas (formatos) específicas para los siguientes tipos de datos:
- Moneda:** \$ 000,000,000.00
- Fecha:** dd/mm/aaaa.
- Hora:** hh:mm:ss (formato de 24 horas).
- Fecha y hora:** dd/mm/aaaa hh:mm:ss (formato de 24 horas).
- 8.3.19 Si se requiere utilizar variables de configuración introducirlas en el archivo de configuración del proyecto o en Base de Datos, evitando manejar datos de código duro en la programación.

8.4. Manejo de Errores

En esta sección se especifican los estándares para el manejo de errores en cada capa del proyecto.

- 8.4.1 Considerar el manejo de un objeto que permita conocer el resultado de la respuesta y el mensaje de error cuando aplique, por ejemplo:
- ```
0 referencias
Public class Resultado
{
 0 referencias
 public int ProcesoExitoso { get; set; }

 0 referencias
 public string Mensaje { get; set; }
}
```
- 8.4.2 Usar las sentencias try y catch para para el manejo y personalización de los errores.
- 8.4.3 Crear un catálogo de errores que nos permita personalizar el manejo de las excepciones, debiendo contener al menos la clave, error, mensaje personalizado (en español) e identificador único global del error (GUID).



- 8.4.4 Registrar los errores en un archivo de texto, el cual debe inicializarse en las capas de Front End y Back End (API). Considerar tener al menos un archivo por día para resguardar los errores que ocurren en la aplicación.
- 8.4.5 Presentar al usuario los errores ocurridos en la aplicación a través de una descripción clara y en español.

## 8.5. Consideraciones para Proyecto Web

En esta sección se especifican las consideraciones para desarrollar un Front End seguro tipo Web, controlando el nivel de acceso a través de roles que permiten la ejecución de las funcionalidades de cada componente.

- 8.5.1 Utilizar el mecanismo de autenticación basado en Token proporcionado por el API en el Front End y Back End.
- 8.5.2 Utilizar los decoradores ALLOWANONYMOUS para los métodos que no necesitan autenticación.
- 8.5.3 Utilizar los decoradores AUTHORIZE para los métodos que sí requieren autenticación.
- 8.5.4 Especificar el nivel de acceso para cada rol a través de los decoradores AUTHORIZE y el atributo ROLES.
- 8.5.5 Inicializar en cada controlador el LOG para registrar los eventos de errores.
- 8.5.6 Utilizar en cada método el manejo de excepción a través de TRY, CATCH y FINALLY.
- 8.5.7 Especificar en cada método de los controladores el tipo de petición que resuelve, por ejemplo: HTTPGET y HTTPPOST.
- 8.5.8 Utilizar los componentes y librerías que la GTI recomienda para el manejo de mensajes, funcionalidades, etc.
- 8.5.9 Utilizar las bondades que nos proporciona RAZOR, el cual forma parte del Framework .Net 8.0.
- 8.5.10 Las páginas deberán contener únicamente los componentes que se utilicen para su funcionalidad.

## 8.6. Consideraciones para Proyecto Windows

En esta sección se especifican las consideraciones para desarrollar un Front End (servicio automático) seguro tipo Windows.

- 8.6.1 Desarrollar la lógica de negocio en el Back End siguiendo la arquitectura definida en este documento.

## 8.7. Consideraciones para Tareas Automatizadas

En esta sección se definen las características a considerar para desarrollar una tarea automatizada.

- 8.7.1 Programar las tareas automatizadas a través de los siguientes mecanismos, dependiendo de la plataforma utilizada:
  - 8.7.1.1 Servicio Windows
  - 8.7.1.2 Job en Base de Datos
  - 8.7.1.3 Cron
- 8.7.2 Desarrollar la lógica de negocio en el Back End siguiendo la arquitectura definida en este documento.



## 8.8. Consideraciones para Controladores en API

En esta sección se especifican las consideraciones para el manejo de Microservicios o Web API.

- 8.8.1 Contar con un método de autenticación que nos permita generar el Token para establecer comunicación con todas las funcionalidades.
- 8.8.2 Utilizar los decoradores ALLOWANONYMOUS para los métodos que no necesitan autenticación.
- 8.8.3 Utilizar los decoradores AUTHORIZE para los métodos que sí requieren autenticación.
- 8.8.4 Especificar el nivel de acceso para cada rol, a través de los decoradores AUTHORIZE y los ROLES.
- 8.8.5 Inicializar en cada controlador el LOG para registrar los eventos de errores.
- 8.8.6 Utilizar en cada método el manejo de excepción a través de las sentencias TRY, CATCH y FINALLY.
- 8.8.7 Especificar en cada método de los controladores el tipo de petición que resuelve, por ejemplo: HTTPGET y HTTPPOST.

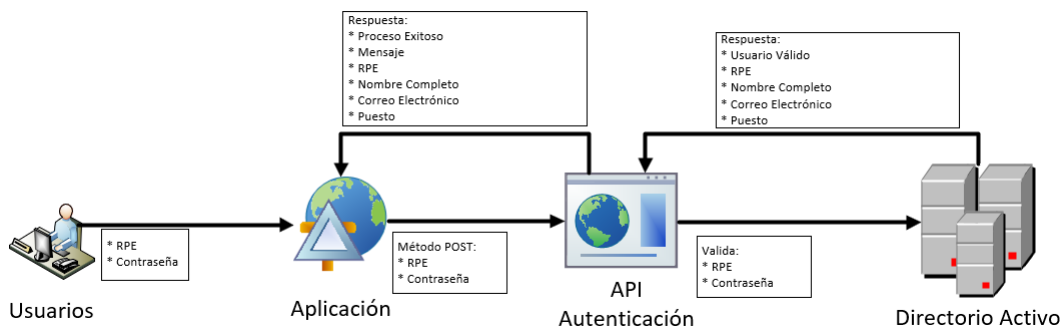
## 8.9. Conexión a Directorio Activo

En esta sección se describe el componente de validación de credenciales a través de Directorio Activo, el cual deberá implementarse en los aplicativos desarrollados a través de los siguientes mecanismos:

- API de Autenticación (realiza una conexión a Directorio Activo).
- Plataforma de identidad.

### 8.9.1 API de Autenticación

El API que permite realizar la validación a través de Directorio Activo es el **API REST**, este debe ser consumido en el método POST de una petición HTTPS. A continuación, se describe el proceso de implementación del API:



El API se deberá configurar con los siguientes parámetros:

- **Curl:**

```
curl -X 'POST' \
 'https://apicst.cfemex.com/APIAutenticacion/api/Autenticacion/Validar' \
 -H 'accept: */*' \
 -H 'Content-Type: application/json' \
 -d '{
 "rpe": "1B808",
 "contrasenia": "MiContrasenia"
 }'
```

- **Método:** POST
- **Dirección:** <https://apicst.cfemex.com/APIAutenticacion/api/Autenticacion/Validar>
- **Parámetros de Entrada:** RPE y contraseña

Al realizar la configuración anterior, se obtendrán los siguientes parámetros de salida:

| No | Parámetro      | Tipo de Dato | Descripción                                                                               |
|----|----------------|--------------|-------------------------------------------------------------------------------------------|
| 1  | Mensaje        | Cadena       | Si las credenciales no son válidas, indicará el mensaje de error.                         |
| 2  | Objeto.Correo  | Cadena       | Si las credenciales son válidas contiene el correo electrónico del empleado.              |
| 3  | Objeto.Nombre  | Cadena       | Si las credenciales son válidas contiene el nombre completo del empleado.                 |
| 4  | Objeto.Puesto  | Cadena       | Si las credenciales son válidas contiene el nombre completo del empleado.                 |
| 5  | Objeto.RPE     | Cadena       | Si las credenciales son válidas contiene el RPE del empleado.                             |
| 6  | ProcesoExitoso | Número       | Indica 1 (Uno) si las credenciales son correctas y 0 (Cero) si no se encontró el usuario. |

En caso de que el RPE y la contraseña proporcionados no sean válidos, se recibirá la siguiente respuesta:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <https://apicst.cfemex.com/APIAutenticacion/api/Autenticacion/Validar>
- Body (JSON):**

```
{
 "rpe": "1B808",
 "contrasenia": "MiContrasenia"
}
```
- Status:** 200 OK
- Time:** 3.74 s
- Size:** 296 B
- Response Body (JSON):**

```
{
 "procesoExitoso": 0,
 "mensaje": "Usuario y/o contraseña incorrectos en Directorio Activo, verifique",
 "objeto": null
}
```

Si el usuario y la contraseña son correctos, se recibirá la siguiente respuesta:

```

POST https://apicst.cfmex.com/APIAutenticacion/api/Autenticacion/Validar
{
 "rpe": "18808",
 "Contraseña": "MiContraseña"
}

{
 "procesoExitoso": 1,
 "mensaje": "",
 "objeto": {
 "rpe": "18808",
 "correo": "erick.garcia@cfemex.com",
 "nombre": "ERICK GARCIA DE LA ROSA",
 "puesto": "AUXILIAR TECNICO"
 }
}

```

### 8.9.2 Plataforma de Identidad

Una plataforma de identidad es una herramienta que permite autenticar a los usuarios, autorizar el acceso a los recursos y gestionar la información relacionada. Al emplear este método de autenticación se podrá utilizar cualquier plataforma de identidad con la que cuente el área en la que se desea implementar la autenticación por Directorio Activo.

## 8.10. Protocolo de Tiempo de Red (NTP)

El equipo en el que se aloje las aplicaciones y bases de datos deberán hacer uso del servicio proporcionado por la GTI para sincronizar la fecha y hora del sistema operativo, asegurando que todos los dispositivos en una red tengan una referencia de tiempo común y precisa.

Esta información la podrá consultar en los catálogos y servicios institucionales provistos por la GTI, mismos que se especifican en la sección de Servicios Tecnológicos de la Intranet Corporativa:

Normatividad TIC / Estándares Tecnológicos / Servicios y Catálogos Institucionales.pdf

## 9. Lineamientos para Base de Datos.

En esta sección se definen los lineamientos de programación para estandarizar la creación de los objetos de las bases de datos de los sistemas desarrollados por y para la CFE.

### 9.1 Nomenclatura de los objetos de las bases de datos.

La definición de los nombres de las bases de datos, esquemas, objetos, vistas, funciones, tablas y atributos deberá apegarse a los siguientes lineamientos:

- 9.1.1 Los nombres de los objetos de la Base de Datos deben comenzar con un prefijo que haga referencia al tipo de objeto del que se trata (tablas, procedimientos almacenados, funciones, vistas, etc.) y utilizar nombres descriptivos en español

que resulten intuitivos y permitan entender su función, la primera letra deberá estar en mayúsculas y las demás en minúsculas, no se deben usar acentos, ni caracteres especiales, como por ejemplo (\$, #, \*, – +, ´, ´´). A excepción de las tablas de tipo “Relación” que llevan guion bajo entre cada tabla.

- 9.1.2 No utilizar espacios en blanco en los nombres de los objetos, esto incluye nombres de columnas.
- 9.1.3 Las bases de datos y tablas deberán definirse en plural.
- 9.1.4 Los nombres de los atributos deberán definirse en singular.
- 9.1.5 La longitud máxima del nombre de un objeto podrá ser de 40 caracteres.
- 9.1.6 No usar palabras reservadas, ni nombres de un único carácter.
- 9.1.7 Al crear el nombre de un objeto compuesto por dos palabras, el primer carácter de cada palabra deberá ir en mayúsculas y colocar la palabra más representativa al inicio, por ejemplo: ConcursoOferta.
- 9.1.8 Toda la información de tipo alfanumérica deberá almacenarse en la base de datos en mayúsculas.

Los tipos de objetos de las bases de datos deberán definirse como se especifica a continuación:

| No | Objeto        | Tipo          | Definición                                                                                                                | Prefijo   | Ejemplo                                                              |
|----|---------------|---------------|---------------------------------------------------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------|
| 1  | Base de datos | Relacional    | Son aquellas que permiten organizar la información en tablas estableciendo relaciones entre ellas.                        | N/A       | Contrataciones, Quejas, Proyectos.                                   |
| 2  | Tablas        | Transaccional | Son aquellas tablas que contienen registros transaccionales.                                                              | tb        | tbOfertas, tbPrestamos, tbSolicitudes, tbRegistrosVencidos.          |
| 3  |               | Catálogo      | Tabla que contiene la información de un catálogo específico. Contienen información de referencia, no son transaccionales. | ct        | ctTipoProcedimientos, ctRoles, ctUsuarios.                           |
| 4  |               | Relación      | Tabla que permite relacionar registros entre dos tablas.                                                                  | No aplica | Tabla 1: ctUsuarios, Tabla 2: ctRoles, Relación: ctUsuarios_ctRoles. |
| 5  |               | Bitácora      | Tabla que permite almacenar información sobre las transacciones efectuadas.                                               | bt        | btUsuarios, btSolicitudes, btOfertas.                                |

|    |                          |               |                                                                                                                  |       |                                             |
|----|--------------------------|---------------|------------------------------------------------------------------------------------------------------------------|-------|---------------------------------------------|
| 6  |                          | Temporal      | Tablas que almacenan información de manera temporal.                                                             | te    | telImportacionSolicitudes, teCalculoNomina. |
| 7  | Vistas                   | N/A           | Representación virtual de los datos de una o más tablas.                                                         | vw    | vwReportesSinAsignar.                       |
| 8  | Procedimiento Almacenado | Consulta      | Procedimiento almacenado que contiene una instrucción SELECT.                                                    | spGet | spGetEmpleados.                             |
| 9  |                          | Inserción     | Procedimiento almacenado que contiene una instrucción INSERT.                                                    | spIns | spInsSolicitudes.                           |
| 10 |                          | Actualización | Procedimiento almacenado que contiene una instrucción UPDATE.                                                    | spUpd | spUpdSolicitudes.                           |
| 11 |                          | Eliminación   | Procedimiento almacenado que contiene una instrucción DELETE.                                                    | spDel | spDelSolicitudes.                           |
| 12 | Función                  | N/A           | Conjunto de funciones predefinidas que realizan una tarea específica.                                            | fn    | fnConcatenar.                               |
| 13 | Triggers                 | N/A           | Conjunto de instrucciones que se ejecutan automáticamente en respuesta a ciertos eventos.                        | tg    | tgActualizaImporte.                         |
| 14 | Jobs                     | N/A           | Tarea programada que se ejecuta automáticamente en un momento específico o en respuesta a un evento predefinido. | jb    | jbLimpiaTemporales.                         |

## 9.2 Tablas

9.2.1 Normalizar el uso de las tablas que conforman la base de datos hasta llegar mínimo a la tercera forma normal.

- 9.2.2 Toda tabla creada deberá contener una llave primaria, la cual deberá ser secuencial, con tipo de dato numérico y entero (tinyint, smallint, int, bigint), y auto-generadas desde el Gestor de Base de Datos.
- 9.2.3 Utilizar el prefijo “id” en la definición del atributo de la llave primaria, por ejemplo: IdEmpleado.
- 9.2.4 No se deben utilizar tipos de datos cadena como llaves primarias.
- 9.2.5 Crear llaves foráneas (FK) que permitan mantener la integridad de la información.
- 9.2.6 Crear catálogos específicos para normar la información del aplicativo, evitando la redundancia en la información, se debe utilizar el identificador único de registro del catálogo en las tablas con las que se relaciona (normalización).
- 9.2.7 Los atributos deberán generarse en orden, comenzando con la llave primaria, seguida de las llaves foráneas y a continuación los atributos de esta con base en su relevancia.
- 9.2.8 Utilizar los tipos de datos de manera adecuada, en concordancia con la información que se almacenará en las tablas. Por ejemplo: fechas, textos, números, etc.
- 9.2.9 Contener las reglas (CONSTRAINTS) necesarias en las columnas donde se requiera ciertas restricciones e ir de acuerdo con los datos que deban almacenarse. Por ejemplo, crear un CONSTRAINT UNIQUE cuando el dato sea único e irrepetible.
- 9.2.10 Las columnas de tipo alfanumérico deberán definirse con el tipo de dato VARCHAR; en caso de no conocer la longitud exacta del campo, utilizar el tipo de dato MAX.
- 9.2.11 En las columnas que se utilicen en las consultas se les deberá generar índices con la finalidad de optimizar las búsquedas.
- 9.2.12 Utilizar el tipo de dato bit para columnas que contienen datos bandera y establecer un valor por Default.

### 9.3 Procedimientos Almacenados

- 9.3.1 Crear procedimientos almacenados independientes para mantener separadas las cuatro operaciones que pueden realizarse en una base de datos: inserción, actualización, consulta y borrado.
- 9.3.2 Nombrar los procedimientos almacenados de acuerdo con la operación que realizan.
- 9.3.3 Incluir siempre la sentencia SET NOCOUNT ON en las operaciones de SELECT, INSERT, UPDATE y DELETE con el objetivo de optimizar las operaciones, a menos, que nos interese saber cuántos registros fueron afectados.
- 9.3.4 No utilizar en las sentencias SELECT el asterisco, nombrar las columnas necesarias para realizar la consulta, a menos que se vayan a utilizar todas las columnas del SELECT.
- 9.3.5 Utilizar la sentencia ORDER BY cuando se consulta información que será presentada al usuario en pantalla. El orden deberá determinarse de acuerdo con la importancia o generalidad de los datos.
- 9.3.6 No utilizar cursores.
- 9.3.7 No utilizar el nombre de la base de datos en las sentencias. Por ejemplo, si se está en la base de datos Operaciones, **NO** será necesario especificar lo siguiente en la sentencia SELECT:

SELECT IdUsuario, Usuario FROM Operaciones.ctUsuarios

Evitando que, si se cambia el nombre de la base de datos, por cualquier razón, no se tenga que cambiar el nombre de la base de datos en todos los procedimientos en los que se haga la referencia.

9.3.8 Evitar en lo posible utilizar sentencias dinámicas, es decir, sentencias que se construyen como cadenas y que se ejecutan mediante la sentencia EXEC.

9.3.9 Inicializar las variables que no funjan como parámetros de entrada.

9.3.10 Escribir todas las palabras reservadas en mayúsculas en las sentencias SQL, por ejemplo:

**SELECT** IdUsuario, Usuario **FROM** ctUsuarios;

**SELECT MAX**(IdUsuario) **FROM** ctUsuarios;

**SELECT** Usuario **FROM** ctUsuarios **ORDER BY** IdUsuario **DESC**;

9.3.11 Documentar todos los procedimientos almacenados. Indicar nombre, fecha, autor, descripción y revisiones.

A continuación, se muestra un ejemplo:

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
/*****
PROCEDIMIENTO: spGetContraOfertasBy
FECHA: 30/05/2018
AUTOR: Erick Garcia de la Rosa
DESCRIPCION: Obtiene las contra ofertas de un proveedor
REVISIONES:
Version Fecha Autor Descripcion

1.0.0 30/05/2018 Erick Garcia de la Rosa Creación del procedimiento.
1.0.1 03/08/2018 Erick Garcia de la Rosa Se adiciona relación a la tabla Procedimiento.
*****/
CREATE PROCEDURE [dbo].[spGetContraOfertasBy]
 @IdProveedor INT
AS
BEGIN
 SET NOCOUNT ON;

 SELECT DISTINCT
 CO.IdProcedimiento,
 P.Numero
 FROM tbContraOferta CO
 INNER JOIN tbProveedorContraOferta PCO ON PCO.IdContraOferta=CO.IdContraOferta
 INNER JOIN tbProveedorProcedimiento PP ON PP.IdProveedorProcedimiento=PCO.IdProveedorProcedimiento
 INNER JOIN tbProcedimiento P ON P.IdProcedimiento=CO.IdProcedimiento
 WHERE PP.IdProveedor=@IdProveedor;
```

## 9.4 Funciones

Documentar todos los procedimientos almacenados. Indicar nombre, fecha, autor, descripción y revisiones.



A continuación, se muestra un ejemplo:

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
/*****
PROCEDIMIENTO: fnConvertirCSVaTabla
FECHA: 30/05/2018
AUTOR: Erick Garcia de la Rosa
DESCRIPCION: Convierte una cadena separada por comas a tabla.
REVISIONES:
Version Fecha Autor Descripcion

1.0.0 30/05/2018 Erick Garcia de la Rosa Creación de la función.
*****/
CREATE FUNCTION [dbo].[fnConvertirCSVaTabla]
(
 @IdProveedor INT
)
RETURNS TABLE
AS
RETURN

AQUÍ VA EL CONTENIDO DE LA FUNCIÓN

END
```

## 9.5 Reportes

Generar los reportes a través de las herramientas LOCAL REPORT o REPORTING SERVICES, la cuales nos permiten crear reportes personalizados a través de plantillas, así como, agilizar el desarrollo para exportar los reportes a PDF, Excel, Word y Html.

Para el desarrollo de los reportes, tomar en consideración los siguientes puntos:

- 9.5.1 El nombre del reporte debe ser descriptivo debiendo referenciar el sistema donde se utiliza y su función. Por ejemplo: SEC\_Concursos.rdlc, EnajenacionBienMuebles\_BienesAdjudicados.rdlc.
- 9.5.2 Considerar el tipo de dato adecuado, a efecto de evitar transformaciones incongruentes en los datos.
- 9.5.3 Los parámetros que se envían deben ser idénticos a los DATASETS establecidos en el reporte.

Todo sistema previo a su liberación deberá pasar por las etapas de aseguramiento de calidad y liberación, tal como se especifica en la descripción del servicio de Desarrollo de Soluciones Tecnológicas, información ubicada en:

Intranet / Servicios Tecnológicos / Servicios Informática (menú superior) / Desarrollo de Soluciones Tecnológicas

Se debe documentar la funcionalidad del job, especificando el periodo de ejecución y los comandos que ejecuta.

## 10 Análisis de vulnerabilidades y revisión del código.

Previo a la liberación del código en ambiente productivo, se deberá revisar el sistema con las herramientas definidas por el área correspondiente de la CST, con el objetivo de realizar un análisis estático de código fuente, así como un análisis de vulnerabilidades.

Adicionalmente, se deberá evaluar el rendimiento y seguridad de los sistemas desarrollados, realizando pruebas de estrés a través de la herramienta JMeter.

## 11 Documentación del sistema.

La documentación técnica inherente al ciclo de implementación y desarrollo de un sistema debe considerar al menos los siguientes documentos:

### 11.1 Inicio (al menos uno de los siguientes documentos):

- Solicitud del requerimiento.
- Acta de proyecto.

### 11.2 Planeación:

- Plan de trabajo.

### 11.3 Ejecución:

#### 11.3.1 Análisis y diseño (al menos uno de los siguientes documentos):

- Especificaciones del proyecto.
- Documento de análisis y diseño.
- Diagrama de procesos o flujo.
- Diagrama de base de datos.
- Maqueta.
- Historias de Usuario.
- Casos de Uso.
- Diagrama de arquitectura.
- Matriz de usuarios/roles.

#### 11.3.2 Aseguramiento de Calidad (al menos uno de los siguientes documentos):

- Documento de pruebas (script de pruebas).
- Matriz de pruebas.
- Resultados de las pruebas piloto.

#### 11.3.3 Implementación (al menos uno de los siguientes documentos):

- Minuta de reunión y/o correo de liberación.
- Entregables.
- Estrategia de implementación.
- Plan de capacitación.

#### 11.3.4 Mantenimiento:

- Control de cambios por mantenimientos, mejoras o hallazgos para mejorar la funcionalidad del sistema.

### 11.4 Monitoreo y control (al menos uno de los siguientes documentos):

- Presentaciones de avances.
- Informes o notas de avance.
- Minutas de reuniones y/o correos de seguimiento.
- Control de cambios (cambios de alcance del proyecto).

### 11.5 Cierre:

- Acta de cierre.
- 11.6 Manuales (al menos uno de los siguientes documentos):**
- Manual Técnico.
  - Manual de Usuario.
  - Infografías.

Cabe mencionar que también se puede considerar cualquier documentación adicional generada durante el ciclo de desarrollo del proyecto.